

Lesson 4 — Put the MCP order assistant behind FastAPI

Goal: turn Lesson 3 into a small, real HTTP application with clear API, service, repository, validation, and dependency-injection boundaries.

The finished application is in `lesson_4_app/`. The notebook explains only the decisions needed to understand and run it.

0. Setup

The application still needs the Lesson 2 MCP server. FastAPI listens on port `8001`; the MCP server listens on `8000`.

```
In [1]: # Run once if needed
%pip install -qU fastapi==0.141.1 langchain-core==1.6.3 langchain-mcp-adapte
```

Note: you may need to restart the kernel to use updated packages.

Require the OpenAI key immediately

The notebook and application fail early when `.env` or `OPENAI_API_KEY` is missing. A configuration error should not wait until the first customer request.

```
In [2]: from pathlib import Path

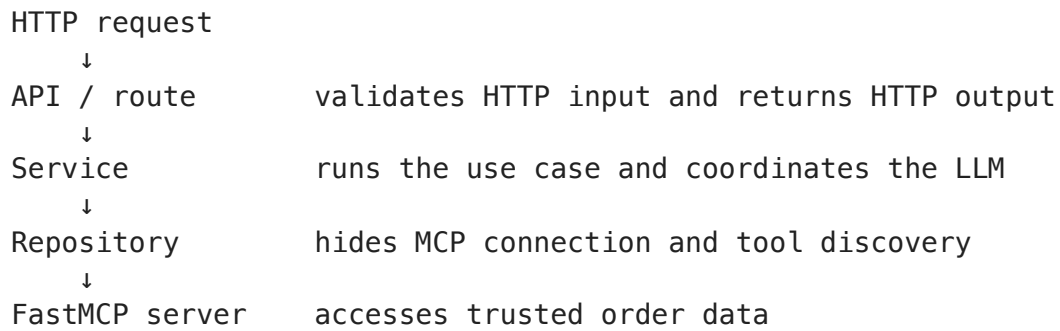
from dotenv import dotenv_values, load_dotenv

dotenv_path = Path.cwd() / ".env"
if not dotenv_path.is_file():
    raise FileNotFoundError(
        f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
    )
if not dotenv_values(dotenv_path).get("OPENAI_API_KEY"):
    raise RuntimeError(f"OPENAI_API_KEY is missing or empty in {dotenv_path}")
load_dotenv(dotenv_path, override=True)
print("Environment loaded.")
```

Environment loaded.

1. Layered architecture—quickly

A layer has one reason to change and depends inward on application logic:



Layer	Contains	Must not contain
API/route	URL, HTTP method, request/response models	Prompt orchestration or MCP transport code
Service	The answer order question use case	FastAPI request objects
Repository	External data/tool access	HTTP routing or user-facing response wording
Schema	Typed boundary contracts	Business workflows

For a tiny app these could fit in one file. We separate them here because the boundaries are the lesson; each file remains small.

Minimal project structure

```

lesson_4_app/
├── app/
│   ├── main.py           # creates FastAPI and includes the router
│   ├── api/routes.py     # HTTP endpoint
│   ├── schemas.py        # request and response contracts
│   ├── services.py       # LLM + tool-calling use case
│   ├── prompt_loader.py  # loads and validates YAML prompts
│   ├── prompts/
│   │   └── order_assistant.yml
│   ├── repositories.py   # MCP boundary
│   ├── dependencies.py   # object construction with Depends
│   └── config.py         # strict environment loading
└── requirements.txt

```

FastAPI's `APIRouter` keeps related path operations in a module, while `app.include_router(...)` assembles them into one application.

2. Transform Lesson 3 into these layers

Lesson 3 responsibility	Lesson 4 location
SupportAnswer	schemas.py
Question input	OrderQuestion in schemas.py

Lesson 3 responsibility	Lesson 4 location
MCP connection and discovery	<code>OrderToolRepository</code>
Prompt text	<code>prompts/order_assistant.yml</code>
Prompt loading and validation	<code>prompt_loader.py</code>
Model tool choice, tool result, structured answer	<code>OrderAssistantService</code>
Object creation	<code>dependencies.py</code>
Public HTTP endpoint	<code>api/routes.py</code>
Application assembly	<code>main.py</code>

Nothing important is rewritten: the Lesson 3 workflow is moved behind stable boundaries.

Compatibility choices used here

- `ChatOpenAI(use_responses_api=True)` selects OpenAI's Responses API explicitly, which is the recommended tool-calling path and avoids endpoint ambiguity.
- Shared model construction does not send model-specific `reasoning_effort` or `temperature` arguments. This keeps both GPT-4o and reasoning models valid; add such controls only after selecting a compatible model.
- `langchain-mcp-adapters` and `MultiServerMCPClient` provide the stable LangChain/MCP boundary. The older `langchain.mcp.MCPAdapter` path is beta.
- Exact dependency versions make the notebook and Docker image reproducible. Upgrade them intentionally and retest the `/ask` flow.

Input and output validation

There are three validation boundaries:

1. **HTTP input:** `OrderQuestion` rejects missing, blank, too-short, or oversized questions before the service runs. Invalid requests receive FastAPI's 422 response.
2. **LLM output:** `with_structured_output(SupportAnswer)` asks the model for the exact Pydantic schema and parses it into that model.
3. **HTTP output:** `response_model=SupportAnswer` makes FastAPI validate, document, serialize, and filter the public response.

Do not trust free-form model text as an API contract. Validate model output before it crosses the HTTP boundary.

Keep prompts outside Python

Production prompts change independently from orchestration code.

`app/prompts/order_assistant.yml` therefore owns the system and final-answer instructions. This makes prompt reviews and version-control diffs focused and avoids redeploy-oriented code edits for wording changes.

The application uses `yaml.safe_load` —never the unsafe generic loader—and validates the result with `OrderPrompts`. `get_order_prompts()` is cached and called during startup, so a missing file, malformed YAML, or missing prompt field fails before traffic is accepted. The validated object is then injected into `OrderAssistantService`.

```
In [3]: from typing import Literal

        from pydantic import BaseModel, ConfigDict, Field

        class OrderQuestion(BaseModel):
            model_config = ConfigDict(str_strip_whitespace=True)
            question: str = Field(min_length=3, max_length=500)

        class SupportAnswer(BaseModel):
            answer: str = Field(min_length=1)
            status: Literal["shipped", "processing", "not_found", "unknown"]
            grounded: bool
```

3. Dependency injection: why, then how

A route needs an `OrderAssistantService`, but it should not know how to create an OpenAI client, load prompts, read configuration, or connect to MCP. **Dependency injection (DI)** lets the route declare what it needs; FastAPI builds and supplies it.

Why it matters:

- construction exists in one place;
- shared configuration is not repeated;
- FastAPI resolves the dependency graph once per request;
- tests can replace the service or repository without calling OpenAI or MCP.

`Depends(get_order_service)` tells FastAPI **how to obtain the value**.

`Annotated[OrderAssistantService, ...]` preserves its Python type for editors and type checkers. A reusable alias keeps route signatures short.

```
In [ ]: from typing import Annotated

from fastapi import Depends

# FastAPI calls this provider; the route does not call it.
def get_order_service() -> OrderAssistantService:
    ...

OrderServiceDep = Annotated[
    OrderAssistantService,
    Depends(get_order_service),
]

# The real route can now state only what it needs:
# async def ask_order(request: OrderQuestion, service: OrderServiceDep): ...
```

4. The complete minimal application

The real implementation is in `lesson_4_app/`; it contains no placeholder application code. Start the MCP server and API from the repository root in separate terminals:

```
python order_mcp_server.py
uvicorn lesson_4_app.app.main:app --reload --port 8001
Open http://127.0.0.1:8001/docs , or send:

curl -X POST http://127.0.0.1:8001/api/v1/orders/ask \
  -H 'Content-Type: application/json' \
  -d '{"question": "What is happening with order A100?"}'
```

References

- [FastAPI: bigger applications and APIRouter](#)
- [FastAPI: dependencies and Annotated](#)
- [FastAPI: response models](#)
- [OpenAI Responses API](#)

5. All required code in one overview

This final cell prints the actual application files in dependency order. Edit the files—not a duplicate notebook copy.

```
In [5]: from pathlib import Path
```

```
app_root = Path("lesson_4_app")
required_files = [
    "app/schemas.py",
    "app/config.py",
    "app/prompts/order_assistant.yml",
    "app/prompt_loader.py",
    "app/repositories.py",
    "app/services.py",
    "app/dependencies.py",
    "app/api/routes.py",
    "app/main.py",
    "requirements.txt",
]

for relative_path in required_files:
    path = app_root / relative_path
    if not path.is_file():
        raise FileNotFoundError(path)
    print(f"\n{'=' * 16} {path} {'=' * 16}\n")
    print(path.read_text())
```

```
===== lesson_4_app/app/schemas.py =====
```

```
from typing import Literal
```

```
from pydantic import BaseModel, ConfigDict, Field
```

```
class OrderQuestion(BaseModel):
```

```
    """Validated HTTP input."""
```

```
    model_config = ConfigDict(str_strip_whitespace=True)
```

```
    question: str = Field(min_length=3, max_length=500)
```

```
class SupportAnswer(BaseModel):
```

```
    """Validated LLM and HTTP output."""
```

```
    answer: str = Field(min_length=1)
```

```
    status: Literal["shipped", "processing", "not_found", "unknown"]
```

```
    grounded: bool = Field(
```

```
        description="True only when the status came from a successful MCP to  
ol call"
```

```
)
```

```
===== lesson_4_app/app/config.py =====
```

```
import os
```

```
from dataclasses import dataclass, field
```

```
from functools import lru_cache
```

```
from pathlib import Path
```

```
from dotenv import dotenv_values, load_dotenv
```

```
@dataclass(frozen=True)
```

```
class Settings:
```

```
    openai_api_key: str = field(repr=False)
```

```
    mcp_url: str = "http://127.0.0.1:8000/mcp"
```

```
    model: str = "gpt-5.6-luna"
```

```
@lru_cache
```

```
def get_settings() -> Settings:
```

```
    dotenv_path = Path(__file__).resolve().parents[2] / ".env"
```

```
    if not dotenv_path.is_file():
```

```
        raise FileNotFoundError(
```

```
            f"Missing {dotenv_path}. Create it with OPENAI_API_KEY=your-key."
```

```
)
```

```
    values = dotenv_values(dotenv_path)
```

```
    api_key = values.get("OPENAI_API_KEY")
```

```
    if not api_key:
```

```
        raise RuntimeError(f"OPENAI_API_KEY is missing or empty in {dotenv_p
```

```
ath}.)
```

```
# Runtime environment variables (for example, Compose configuration) with
n.
load_dotenv(dotenv_path, override=False)
return Settings(
    openai_api_key=api_key,
    mcp_url=os.getenv("MCP_URL", "http://127.0.0.1:8000/mcp"),
    model=os.getenv("OPENAI_MODEL", "gpt-5.6-luna"),
)
```

```
===== lesson_4_app/app/prompts/order_assistant.yml =====
==
```

```
system: |
    You are a minimal order-support assistant.
    Use get_order_status whenever an order status is requested.
    Never invent a status. Base the final answer only on a successful tool result.
    If verified information is unavailable, use status='unknown'.

final_answer: |
    Return the final structured answer using only successful tool results.
    If no successful result exists, use status='unknown' and grounded=False.
```

```
===== lesson_4_app/app/prompt_loader.py =====
```

```
from functools import lru_cache
from pathlib import Path
```

```
import yaml
from pydantic import BaseModel, Field
```

```
class OrderPrompts(BaseModel):
    """Validated prompt configuration loaded from YAML."""
```

```
    system: str = Field(min_length=1)
    final_answer: str = Field(min_length=1)
```

```
@lru_cache
def get_order_prompts() -> OrderPrompts:
    prompt_path = Path(__file__).parent / "prompts" / "order_assistant.yml"
    if not prompt_path.is_file():
        raise FileNotFoundError(f"Missing prompt file: {prompt_path}")

    with prompt_path.open(encoding="utf-8") as prompt_file:
        raw_prompts = yaml.safe_load(prompt_file)

    return OrderPrompts.model_validate(raw_prompts)
```

```
===== lesson_4_app/app/repositories.py =====
```

```

from langchain_core.tools import BaseTool
from langchain_mcp_adapters.client import MultiServerMCPClient

```

```

class OrderToolRepository:
    """Data-access boundary for the external order MCP server."""

    def __init__(self, mcp_url: str) -> None:
        self._client = MultiServerMCPClient(
            {
                "orders": {
                    "transport": "http",
                    "url": mcp_url,
                }
            }
        )

    async def get_tools(self) -> list[BaseTool]:
        return await self._client.get_tools()

```

===== lesson_4_app/app/services.py =====

```

from langchain_core.language_models.chat_models import BaseChatModel
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_core.prompts import ChatPromptTemplate

```

```

from .prompt_loader import OrderPrompts
from .repositories import OrderToolRepository
from .schemas import SupportAnswer

```

```

class OrderAssistantService:
    """Application use case: answer one order question with an MCP tool."""

    def __init__(
        self,
        llm: BaseChatModel,
        repository: OrderToolRepository,
        prompts: OrderPrompts,
    ) -> None:
        self._llm = llm
        self._repository = repository
        self._prompts = prompts

    async def answer(self, question: str) -> SupportAnswer:
        prompt = ChatPromptTemplate.from_messages(
            [("system", self._prompts.system), ("human", "{question}")]
        )
        messages = prompt.invoke({"question": question}).to_messages()

        tools = await self._repository.get_tools()
        tool_map = {tool.name: tool for tool in tools}
        model_with_tools = self._llm.bind_tools(tools).with_retry(
            stop_after_attempt=3
        )

```

```

    )
    first_response = await model_with_tools.ainvoke(messages)
    messages.append(first_response)

    for call in first_response.tool_calls:
        tool = tool_map.get(call["name"])
        if tool is None:
            messages.append(
                ToolMessage(
                    content="Tool unavailable",
                    tool_call_id=call["id"],
                    status="error",
                )
            )
        else:
            messages.append(await tool.ainvoke(call))

    messages.append(HumanMessage(content=self._prompts.final_answer))
    structured_llm = self._llm.with_structured_output(SupportAnswer).with_retry(
        stop_after_attempt=3
    )
    return await structured_llm.ainvoke(messages)

```

===== lesson_4_app/app/dependencies.py =====

```

from typing import Annotated

from fastapi import Depends
from langchain_core.language_models.chat_models import BaseChatModel
from langchain_openai import ChatOpenAI

from .config import Settings, get_settings
from .prompt_loader import OrderPrompts, get_order_prompts
from .repositories import OrderToolRepository
from .services import OrderAssistantService

```

```

SettingsDep = Annotated[Settings, Depends(get_settings)]

```

```

def get_llm(settings: SettingsDep) -> BaseChatModel:
    return ChatOpenAI(
        api_key=settings.openai_api_key,
        model=settings.model,
        use_responses_api=True,
    )

```

```

LlmDep = Annotated[BaseChatModel, Depends(get_llm)]

```

```

def get_repository(settings: SettingsDep) -> OrderToolRepository:
    return OrderToolRepository(settings.mcp_url)

```

```
RepositoryDep = Annotated[OrderToolRepository, Depends(get_repository)]
PromptsDep = Annotated[OrderPrompts, Depends(get_order_prompts)]
```

```
def get_order_service(
    llm: LlmDep,
    repository: RepositoryDep,
    prompts: PromptsDep,
) -> OrderAssistantService:
    return OrderAssistantService(llm=llm, repository=repository, prompts=prompts)
```

```
OrderServiceDep = Annotated[OrderAssistantService, Depends(get_order_service)]
```

```
===== lesson_4_app/app/api/routes.py =====
```

```
from fastapi import APIRouter

from ..dependencies import OrderServiceDep
from ..schemas import OrderQuestion, SupportAnswer

router = APIRouter(prefix="/orders", tags=["orders"])

@router.post("/ask", response_model=SupportAnswer)
async def ask_order(
    request: OrderQuestion,
    service: OrderServiceDep,
) -> SupportAnswer:
    return await service.answer(request.question)
```

```
===== lesson_4_app/app/main.py =====
```

```
from fastapi import FastAPI

from .api.routes import router
from .config import get_settings
from .prompt_loader import get_order_prompts

# Fail during startup/import instead of on the first request.
get_settings()
get_order_prompts()

app = FastAPI(title="Lesson 4 Order Assistant", version="1.0.0")
app.include_router(router, prefix="/api/v1")

@app.get("/health")
async def health() -> dict[str, str]:
```

```
        return {"status": "ok"}

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="127.0.0.1", port=8001)

===== lesson_4_app/requirements.txt =====

fastapi==0.141.1
langchain-core==1.6.3
langchain-mcp-adapters==0.3.2
langchain-openai==1.6.2
openai==3.14.1
pydantic==2.13.5
python-dotenv==1.2.3
PyYAML==6.0.3
uvicorn[standard]==0.53.0
```

In []: